Visual Object Search in Drone Images

Ashish Pratap Singh 261010070
Indian Institute of Technology Kanpur
ashishps26@iitk.ac.in

Abstract

Finding every instance of a user-specified object in a high-resolution drone image is difficult: objects cover only a few dozen pixels, scale and appearance vary, and a closed-set detector cannot anticipate new object types. We study query-by-example search*: given a box around an object in a support image, return all instances of that object in a target drone image, with no training. Our pipeline combines frozen DINOv2 patch features computed on tiles of the target image, a rotation-aware query encoding in which the support window is rotated and re-embedded, template matching over a multi-scale feature pyramid, and a coarse-to-fine step that re-matches the top candidates in a local window at twice the resolution. We evaluate on 117 one-shot episodes built from VisDrone-DET, with queries taken from different images. Coarse-to-fine refinement is the most useful component: on top of a fixed pyramid it raises the mean per-episode AP_{50} from 10.6 to 13.6 (paired bootstrap 95% CI of the difference $[+1.6, +4.4]$) and Recall@100 from 29.2 to 31.7. Rotation-aware encoding adds 4.1 points of Recall@100 but no measurable AP gain, and a pyramid whose scales are derived from the query size did not help (it was slightly worse). A tiled OWL-ViT image-guided baseline reaches 2.4–3.5 AP_{50} and is competitive only on large objects. AP rises from 13.5 at IoU 0.5 to 37.8 at IoU 0.1, which shows that box size, inherited from the query, is the main limitation of the current method.*

1. Introduction

Drones are widely used for traffic monitoring, search and rescue, and surveillance. In these settings an operator often wants to ask an ad-hoc question such as "find all vehicles that look like *this one*" in a large aerial image. The standard answer is a supervised detector such as YOLO [1] trained on a fixed label set (for VisDrone [2], ten classes). Such a detector cannot find an object type outside its label set without new labelled data and retraining.

Visual object search instead takes a single example as the query. The aerial setting makes this hard for four reasons: (i) objects are tiny relative to the image (VisDrone images are about 2000×1500 pixels), so downscaling the image erases them; (ii) the same object appears at very different scales depending on altitude; (iii) a top-down camera has no canonical "up", so the query may be rotated relative to the target; and (iv) the query comes from a different image, with different lighting and blur.

Problem statement. *Input:* a target drone image and a query, i.e. a bounding box on a support image. *Output:* a ranked list of boxes in the target image that contain the same kind of object, plus the estimated rotation of the match. *Constraints:* no training or fine-tuning for a new object; one example per query; images of up to several thousand pixels.

Approach. We use frozen DINOv2 [3] patch features and treat search as template matching in feature space. The target is tiled so small objects keep their resolution; the query is encoded after rotating the support window by multiples of $90°$; and the best candidates are re-matched in a local window at twice the resolution.

What is and is not new. The individual ingredients are not new: tiling large aerial images [4, 5], DINOv2 patch matching for few-shot tasks [6–8], and rotated references [7] all have precedent. We do not claim a new backbone or a first-of-its-kind system; our literature search was limited, so we also cannot rule out closely related prior work. Our contributions are:

1. An adaptation of training-free DINOv2 matching to one-shot object search in drone images, with a rotation-aware query encoding that rotates and *re-embeds* the support window (ViT features are not rotation-equivariant), and a coarse-to-fine local refinement step.
2. An episodic evaluation protocol on VisDrone-DET with cross-image queries, and a component-wise ablation with paired bootstrap confidence intervals.
3. Evidence on what does and does not help: coarse-to-fine refinement helps AP and recall; rotation helps recall only; a query-size-derived scale pyramid, which we initially expected to be a contribution, did not help.

2. Related Work

Detection in drone imagery. Supervised detectors such as YOLO [1] are trained on benchmarks like VisDrone [2], and are closed-set. Large aerial images are usually handled by slicing them into tiles and detecting on each; SAHI [4] formalises slicing-aided inference for small objects, and Xie et al. [5] study efficient detection in large aerial images. We adopt tiling but apply it to feature extraction instead of detection.

Few-shot and image-guided detection. Few-shot detectors adapt a detector with a few labelled examples [9]. OWL-ViT [10] supports image-conditioned ("one-shot") detection with a pretrained open-vocabulary model; we use it as our baseline. Bou et al. [6] study robust features for few-shot detection in satellite imagery, which is the closest work to ours in application; we did not compare against it.

Training-free matching with self-supervised features. DINOv2 [3] patch features support dense correspondence without fine-tuning. AnomalyDINO [7] matches patch features against a few reference images and offers rotation of the references, and Espinosa et al. [8] build a training-free reference-based segmentation pipeline on frozen foundation models. Our work follows this training-free line and differs in the target task (box-level search in tiled, high-resolution drone images), the multi-scale kernel matching, and the coarse-to-fine refinement we evaluate.

3. Method

Figure 1 gives an overview. All components use a frozen DINOv2 ViT-B/14 (patch size $P = 14$); nothing is learned.

3.1. Dense features for large images

A target image is resized by a scale $s_k = 2^{k/2}$ (level k, half-octave steps) and split into 518×518 tiles (a 37×37 patch grid) with stride 364 pixels. The patch tokens of overlapping tiles are averaged, stitched into one map, and L2-normalised per patch, giving $F_k \in \mathbb{R}^{C \times h_k \times w_k}$. Levels are computed lazily and cached. Levels whose longer side would exceed 4032 pixels are skipped.

3.2. Rotation-aware query encoding

Let the query be a box (x, y, w, h) on a support image. We set the rescale factor $r_q = n_p P / \max(w, h)$ so the object spans about n_p patches. A context window around the box (half-size $\max(1.5 \max(w, h),\ 112/r_q)$) is cropped and rescaled by r_q. For each rotation $\rho \in \{0, 1, 2, 3\}$ the *window and a box mask* are rotated by $\rho \cdot 90°$ and embedded by DINOv2, and the tokens inside the rotated box are ROI-aligned into a kernel $K_\rho \in \mathbb{R}^{C \times g_h \times g_w}$, which is L2-normalised per cell. Rotating the image before embedding is necessary because ViT features are not rotation-equivariant; it also makes the kernel shape change for odd ρ.

The best-scoring ρ at a detection is returned as the estimated orientation (we do not evaluate orientation accuracy).

3.3. Multi-scale template matching

For level k and rotation ρ, the score map is the mean cosine similarity between the kernel and the feature map,

$$S_{k,\rho}(i, j) = \frac{1}{g_h g_w} \sum_{u=1}^{g_h} \sum_{v=1}^{g_w} \big\langle F_k(i+u, j+v),\ K_\rho(u, v) \big\rangle. \tag{1}$$

Local maxima (3×3 neighbourhood) with $S > 0.25$ become candidates (at most 300 per map). A candidate's box is the (rotated) query box size placed at the kernel centre, divided by the level scale to return to image coordinates; its score is S. Candidates from all levels and rotations are merged with NMS (IoU 0.5, at most 300 kept).

Pyramid policies. *Single scale* uses $k = 0$. *Fixed pyramid* uses $k \in \{-2, \ldots, 2\}$ (scales 0.5–2) for every query. *Query-adaptive pyramid* centres the levels on $k_0 = \mathrm{round}(2 \log_2 r_q)$, i.e. $k \in \{k_0 - 2, \ldots, k_0 + 2\}$, the level at which an object of the query's size spans n_p patches. We tested the adaptive policy as a possible contribution; Sec. 4.3 shows it did not help.

3.4. Coarse-to-fine refinement

The top $M = 150$ candidates are refined. For a candidate at level k we crop a window of half-size $\max(1.5 \max(\text{box side}),\ 64/s_f)$ around its centre, resize it by $s_f = 2s_k$, embed it, and match it with a kernel re-extracted at scale $2r_q$ (same rotation). The match position is searched only within ± 2 fine patches of the coarse centre to avoid jumping to a neighbouring instance. The refined score is $\alpha s_{\text{coarse}} + (1 - \alpha) s_{\text{fine}}$ with $\alpha = 0.5$; unrefined candidates keep αs_{coarse}, so they rank below refined ones. A final NMS gives the output.

4. Experiments

4.1. Data and protocol

Dataset. VisDrone2019-DET [2]: ten object classes plus "ignored regions". Annotations list $(x, y, w, h, \text{score}, \text{category}, \text{truncation}, \text{occlusion})$; boxes with score 0 are not counted and detections centred in ignored regions are discarded.

Episodes. An episode is (target image from the VisDrone *val* split, category c, query from a *different train* image). The task is to find all instances of c in the target. A category is eligible if the target contains at least three countable instances; at most two categories are sampled per image. Queries are clean instances (score 1, not truncated, not occluded, shorter side ≥ 12 px, longer side ≤ 300 px) drawn from every fifth train annotation file. With a fixed seed we obtain **117 episodes on 60 val images**; all methods see the same episodes.

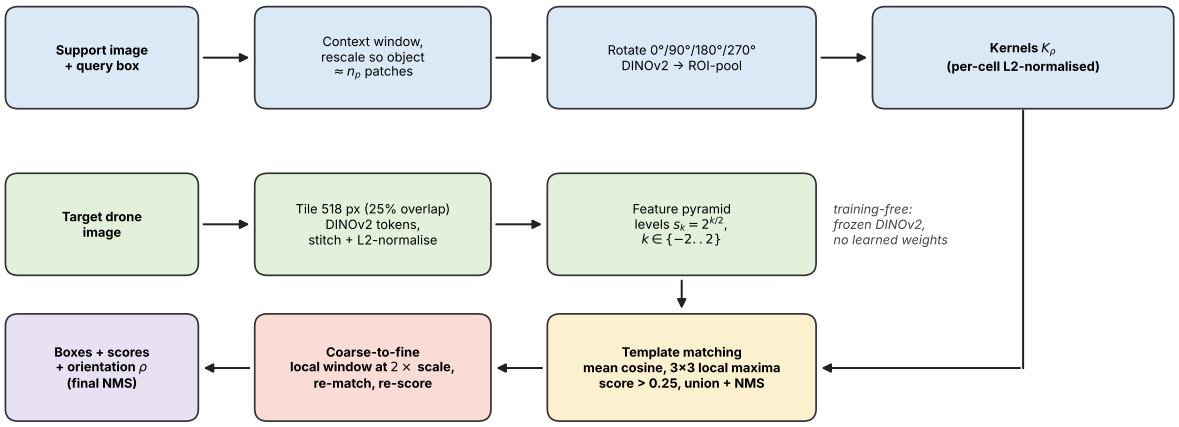


Figure 1. Overview of the pipeline. The query branch (top) turns a box on a support image into rotated patch-feature kernels K_ρ. The target branch (middle) builds a tiled DINOv2 feature pyramid. Kernels are slid over the pyramid to produce coarse candidates, which are refined in local windows at $2\times$ resolution.

Metrics. For each episode we compute AP at IoU 0.5 (all-point interpolation) and Recall@100 (fraction of ground-truth boxes matched by the 100 highest-scoring detections), and report means over episodes; the tables label the AP column AP_{50}. This is a per-episode AP and is *not* the class-averaged mAP of standard detection benchmarks, so the numbers are not comparable with published VisDrone results. We also report the runtime per episode (seconds, including feature extraction for the pyramid levels used, excluding image loading) on a single Kaggle GPU. Sizes follow the median longer box side of the category's ground truth in the target: small (< 32 px), medium (32–64 px), large (> 64 px).

4.2. Implementation details

DINOv2 ViT-B/14 (`facebook/dinov2-base`, fp16), tile 518 px with stride 364 px, $n_p = 4$, $\alpha = 0.5$, refinement of the top 150, score threshold 0.25, NMS IoU 0.5. The value $n_p = 4$ was chosen after inspecting results on a separate tuning set of val images disjoint from the 60 evaluation images (Appendix A). We disclose that an earlier n_p sweep on the evaluation episodes themselves suggested the same value; we discarded it and repeated the sweep on held-out images, where the full pipeline actually scored higher at $n_p = 3$ than at $n_p = 4$, while the non-refined variants preferred n_p of 4 to 5. With only 37 tuning images these differences are small and noisy, so $n_p = 4$ should be read as a reasonable choice, not an optimum.

Baseline. OWL-ViT [10] (`google/owlvit-base-patch32`) image-guided detection, run on 768×768 tiles (stride 576) of the target at $1\times$ and $2\times$ scale, with the query box cropped (15% margin) from the support image, score threshold 0.05 and NMS. We did not tune the baseline, so its numbers may understate what a tuned OWL-ViT could reach.

	Method	AP_{50}	R@100	s/ep
A	Single scale	4.49	8.13	0.16
B	Fixed pyramid	10.64	29.16	1.19
C	Adaptive pyramid	8.97	28.27	1.80
D	C + rotation	8.48	30.31	1.85
E	D + coarse-to-fine	12.51	36.41	3.74
F	B + coarse-to-fine	**13.59**	31.68	2.52
G	F + rotation (final)	13.50	35.80	3.04
	OWL-ViT tiled, $1\times$	2.36	7.28	0.63
	OWL-ViT tiled, $2\times$	3.54	15.29	2.58

Table 1. Mean over 117 episodes (%, runtime in s/episode). A–E were planned; F and G were added afterwards.

Configurations. All use the same encoder and $n_p = 4$: A single scale; B fixed pyramid; C adaptive pyramid; D adaptive pyramid with 4 rotations; E adaptive pyramid, rotations and refinement; F fixed pyramid with refinement; G fixed pyramid with rotations and refinement. **F and G were added after we saw the results of A–E** (refinement looked useful, adaptive scales did not), so choosing G as the final method is a post-hoc decision on the same episodes. Treat results involving F and G as exploratory.

4.3. Results

Main ablation. Table 1 and Fig. 2 show the results, and Table 2 gives paired bootstrap 95% confidence intervals for differences (10,000 resamples, episodes as units, no correc-

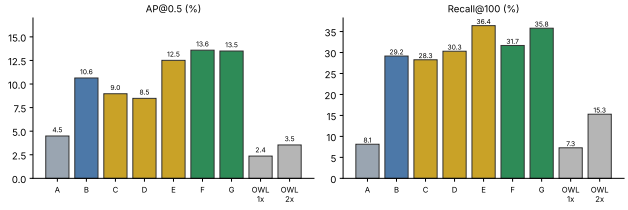


Figure 2. AP_{50} and Recall@100 for each configuration (data of Table 1).

Comparison	ΔAP_{50} [95% CI]	$\Delta R@100$ [95% CI]
B − A (pyramid)	$+6.15$ [$+3.74$, $+8.59$]	–
C − B (adaptive)	-1.66 [-3.37, -0.09]	–
D − C (rotation)	-0.49 [-1.40, $+0.56$]	–
E − D (refine)	$+4.03$ [$+2.53$, $+5.60$]	–
F − B (refine)	$+2.95$ [$+1.59$, $+4.36$]	$+2.52$ [$+0.58$, $+4.49$]
G − F (rotation)	-0.09 [-1.25, $+0.96$]	$+4.12$ [$+1.78$, $+6.57$]
E − B	$+1.87$ [-0.54, $+4.18$]	$+7.25$ [$+2.52$, $+11.98$]
F − E	$+1.07$ [-1.13, $+3.37$]	-4.73 [-9.45, -0.03]

Table 2. Paired differences between configurations (percentage points). A dash means the interval was not computed.

tion for multiple comparisons; episodes from the same image are not independent, so the intervals are indicative).

We draw four conclusions. (1) *Multi-scale search matters*: the fixed pyramid improves AP_{50} by 6.2 points over single-scale matching. (2) *Coarse-to-fine refinement is the most useful component*: on the fixed pyramid it adds 3.0 AP_{50} points and 2.5 recall points, and both intervals exclude zero. (3) *Rotation helps recall but not AP*: on top of F it adds 4.1 Recall@100 points while AP_{50} is unchanged (-0.09, interval spanning zero). (4) *The query-adaptive pyramid did not help*: it was 1.7 points *below* the fixed pyramid, with an interval that only barely excludes zero, so we read it as weak evidence of harm, not as proof. We did not test why; a plausible but unverified explanation is that the size of a query taken from a different image is a noisy estimate of the target object's size.

The full adaptive pipeline E is not significantly different from the plain fixed pyramid B on AP (interval spans zero), although it has much higher recall (+7.25). G has AP and recall comparable to E (36.4 vs. 35.8 Recall@100; we did not test G against E directly) while being faster (3.04 s against 3.74 s) and not depending on the adaptive pyramid, which did not help; this is why we use G as the final method.

Comparison with OWL-ViT. Tiled OWL-ViT scores 2.4 and 3.5 AP_{50} overall. It is competitive only on large objects (16.3 against 17.2 for G, no interval computed) and almost fails on small ones (0.1–2.1 against 12.1), which dominate aerial imagery (Table 3). We caution that the baseline was not tuned and that low-resolution crops of tiny queries may disadvantage it; the result shows that off-the-shelf tiled OWL-ViT is a weak baseline in this setting, not that image-guided detection is inherently weak.

Method	Small	Medium	Large
A Single scale	1.93	8.14	8.23
B Fixed pyramid	8.60	13.63	13.35
C Adaptive	8.17	9.50	12.24
D C + rotation	7.62	9.60	10.18
E D + refine	12.67	11.03	16.53
F B + refine	12.06	15.91	15.33
G F + rotation	12.13	14.93	**17.20**
OWL-ViT 1×	0.10	2.44	16.25
OWL-ViT 2×	2.09	3.91	11.33

Table 3. AP_{50} (%) by object size. Small < 32 px, medium 32–64 px, large > 64 px (median longer box side). We did not compute confidence intervals per bucket.

IoU threshold	0.1	0.3	0.5
AP of G (%)	37.75	26.22	13.50

Table 4. Average precision of the final method G at different IoU thresholds (117 episodes).

Localisation is the main bottleneck. Table 4 shows the AP of G at three IoU thresholds. AP nearly triples when the threshold is relaxed from 0.5 to 0.1, which means many detections fall on or near a correct object but with a box that is not tight enough. Our output box size is the query box size rescaled by the pyramid level, so it is wrong whenever the query object and the target object differ in size. The value at IoU 0.1 is an upper-bound indication only, since in crowded scenes a loose box can match a neighbouring instance. Even at IoU 0.1 AP is 37.8, so detection itself is also limited.

4.4. Qualitative results

Figure 3 shows the highest- and lowest-AP episodes for G. The method works best in dense scenes with objects of the same type and similar scale to the query (for example, cars in a traffic jam). Failure modes visible in our qualitative examples include: (i) *box size mismatch*, where the query object is much larger or smaller than the target objects, so detections are on the right region with wrong box size; (ii) *appearance gap*, with dark, blurred, or overexposed queries or targets; (iii) *class confusion*, where e.g. a truck query returns cars and vans; and (iv) *duplicates*, since NMS at IoU 0.5 does not merge nested boxes of different sizes on one object. The figures show the top 40 detections regardless of score, which makes them look more cluttered than the ranking suggests.



Figure 3. Left: highest-AP episodes of G. Right: lowest-AP episodes of G. In each row, the left panel is the query crop (enlarged for display; the model actually receives the surrounding context window, rescaled so the object spans n_p patches) and the right panel shows the top-40 detections (red) and ground truth (green). These are best and worst cases, not typical ones.

5. Interactive Demo

We provide a Gradio demo (Fig. 4) built on the same code as the experiments (final configuration G). The user uploads a drone image, clicks two opposite corners of an object, and receives the matching boxes with their scores and matched rotation; settings allow switching rotation and refinement on or off and changing the score threshold. The first search on a new image builds the feature pyramid (a few seconds), and later queries on the same image reuse it. Two caveats apply. First, the demo marks the query on the *same* image, which is an easier setting than the cross-image queries of our experiments and was not evaluated. Second, the demo can merge overlapping boxes for display (on by default, switchable); this clean-up is not part of the evaluated pipeline, and no number in this report uses it. Fig. 4 is a single illustrative example, not a result. The demo code is listed in Appendix C.



Figure 4. Demo output for a query on the white car (blue box). Red boxes show matches with score and matched rotation. One example selected by hand, not a measured result.

6. Limitations and Discussion

- **Small evaluation.** 117 episodes on 60 images, one seed, one dataset. Several intervals are wide, and some conclusions (adaptive pyramid, E vs. B on AP) are not firm.
- **Post-hoc selection.** G was selected after seeing results on the same episodes; n_p was tuned on held-out images but with few episodes.
- **Box size is inherited from the query.** This is the largest identified weakness (Table 4). Predicting the box extent from the matched patches is a natural next step but is untested.
- **Cross-image queries.** Query quality varies (blur, darkness, tiny crops), and the scale between support and target differs. The behaviour with same-image queries, as in an

interactive demo, was not evaluated.

- **No supervised reference.** We did not train a closed-set detector for comparison; it would likely score higher on the ten VisDrone classes but cannot search for new objects.
- **Cost.** About 3 s per episode on a GPU for the final method. Orientation is output but not evaluated.

7. Conclusion

We built and evaluated a training-free system for query-by-example object search in drone images using DINOv2 patch matching, tiling, rotation-aware query encoding, and coarse-to-fine refinement. In our experiments, refinement is the component that reliably helps, rotation improves recall, and an adaptive scale pyramid does not help. The main open problem is box localisation. The code and all numbers in this report are reproducible from the appendix and the repository below.

Individual Contribution

This project was carried out individually by Ashish Pratap Singh. I designed the pipeline and experiments, ran all experiments on Kaggle, analysed the results, and wrote this report and the code. I used an AI assistant (Claude, Anthropic) to help draft the report text; I ran all experiments and checked all reported numbers myself.

Links

Code repository (experiments and demo): https://github.com/aash-singh/EE604-Course-Project-Object-Detection-in-Drone-Images
VisDrone dataset: https://github.com/VisDrone/VisDrone-Dataset
DINOv2 code: https://github.com/facebookresearch/dinov2; weights: https://huggingface.co/facebook/dinov2-base
OWL-ViT weights: https://huggingface.co/google/owlvit-base-patch32

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 1, 2

[2] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and tracking meet drones challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2021. 1, 2

[3] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023. 1, 2

[4] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *IEEE International Conference on Image Processing (ICIP)*, 2022. 1, 2

[5] X. Xie, G. Cheng, Q. Li, S. Miao, K. Li, and J. Han, "Fewer is more: Efficient object detection in large aerial images," *arXiv preprint arXiv:2212.13136*, 2022. 1, 2

[6] X. Bou, G. Facciolo, R. G. von Gioi, J.-M. Morel, and T. Ehret, "Exploring robust features for few-shot object detection in satellite imagery," *arXiv preprint arXiv:2403.05381*, 2024. 1, 2

[7] S. Damm, M. Laszkiewicz, J. Lederer, and A. Fischer, "AnomalyDINO: Boosting patch-based few-shot anomaly detection with DINOv2," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025. arXiv:2405.14529. 1, 2

[8] M. Espinosa, C. Yang, L. Ericsson, S. McDonagh, and E. J. Crowley, "No time to train! training-free reference-based instance segmentation," *arXiv preprint arXiv:2507.02798*, 2025. 1, 2

[9] B. Kang, Z. Liu, X. Wang, F. Yu, J. Feng, and T. Darrell, "Few-shot object detection via feature reweighting," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. 2

[10] M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn, A. Dosovitskiy, A. Mahendran, A. Arnab, M. Dehghani, Z. Shen, *et al.*, "Simple open-vocabulary object detection with vision transformers," in *European Conference on Computer Vision (ECCV)*, 2022. 2, 3

A. Hyper-parameter tuning on held-out images

Tuning episodes come from VisDrone val images disjoint from the 60 evaluation images (seed 1). "Fixed" and "adaptive" use one rotation and no refinement; "full" is the adaptive pyramid with 4 rotations and refinement (configuration E).

Setting	AP_{50}	R@100	s/ep
fixed, n_p=3	6.64	26.87	1.19
adaptive, n_p=3	6.55	25.63	1.58
full, n_p=3	12.34	34.98	2.93
fixed, n_p=4	7.90	26.23	1.19
adaptive, n_p=4	8.48	27.98	1.90
full, n_p=4	10.30	34.64	3.90
fixed, n_p=5	7.79	21.01	1.19
adaptive, n_p=5	9.29	26.43	2.00
full, n_p=5	10.84	33.96	5.04
fixed, n_p=6	6.50	18.01	1.19
adaptive, n_p=6	8.62	25.49	2.04
full, n_p=6	10.41	30.09	5.91

Table 5. Tuning results (%). We selected $n_p = 4$; the full pipeline was highest at $n_p = 3$ here.

B. Reproducibility

Models: `facebook/dinov2-base` and `google/owlvit-base-patch32` from the HuggingFace hub. Random seeds: 0 for evaluation episodes, 1 for tuning episodes, 0 for the bootstrap. The OWL-ViT rows of Table 1 come from the first full run, on the same 117 episodes (same seed) as our configurations.

C. Complete code

C.1. Experiments (notebook code)

The complete notebook code (all cells, in the order they were developed) follows. The same file is in the repository.

```
# %% [markdown]
# # Visual Object Search in Drone Images
# **Query-by-example, training-free search with DINOv2 patch features**
#
# Input : a drone image + a query (a box drawn on a support image showing the object of interest)
# Output: boxes (with scores and estimated orientation) of every instance of that object in the image
#
# **Kaggle setup (do this first)**
# 1. Settings -> Accelerator -> GPU (T4 or P100)
# 2. Settings -> Internet -> ON (needed to download DINOv2 / OWL-ViT weights)
# 3. Add Input -> search "VisDrone" -> add a dataset containing VisDrone2019-DET (train + val, with images/ and annotations/ folders)
#
# **Prior work this builds on (cite in the report)**
# * DINOv2 backbone: Oquab et al., 2023 (arXiv 2304.07193)
# * Tiling of large aerial images: standard practice, e.g. SAHI (Akyon et al., 2022) and Xie et al. (arXiv 2212.13136)
# * Reference-based detection with DINOv2 features on remote-sensing images: Bou et al., 2024 (arXiv 2403.05381)
# * Training-free DINOv2 patch matching with rotated references: AnomalyDINO, Damm et al. (arXiv 2405.14529)
# * Training-free reference-based segmentation with DINOv2 + SAM2: Espinosa et al., 2025 (arXiv 2507.02798)
# * Baseline: OWL-ViT image-guided detection, Minderer et al., ECCV 2022
# * Data: VisDrone, Zhu et al., TPAMI 2021
#
# **Our contributions (what is ours vs. borrowed)**
# 1. Query-scale-adaptive pyramid: the target scale levels are derived from the pixel size of the query box, instead of a fixed pyramid.
# 2. Orientation-aware matching: the *support image* is rotated (0/90/180/270) and re-embedded (ViT features are not rotation-equivariant); the best rotation is returned as the object's estimated orientation.
# 3. Coarse-to-fine search: refine only the top candidates at 2x resolution in a local window (sub-patch localisation + re-ranking).
# 4. Systematic ablation of each component on VisDrone against fixed-scale tiling and OWL-ViT.

# %% Imports and global settings
import os, time, json, math, random, glob
from dataclasses import dataclass
from collections import defaultdict

import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
from torchvision.ops import nms, roi_align
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
FEAT_DTYPE = torch.float16 if DEVICE == "cuda" else torch.float32
PATCH = 14                          # DINOv2 patch size
OUT_DIR = "/kaggle/working/results" if os.path.exists("/kaggle") else "./results"
os.makedirs(OUT_DIR, exist_ok=True)
random.seed(0); np.random.seed(0); torch.manual_seed(0)
print("device:", DEVICE)


def sync():
    """Wait for the GPU so that timings are honest."""
    if DEVICE == "cuda":
        torch.cuda.synchronize()


# %% [markdown]
# ## 1. VisDrone loading
# VisDrone-DET annotation line: 'x,y,w,h,score,category,truncation,occlusion'.
# * category 0 = *ignored region*; 1..10 = pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor
# * score = 1 -> counted in evaluation, 0 -> ignored

# %% VisDrone helpers
CLASSES = {1: "pedestrian", 2: "people", 3: "bicycle", 4: "car", 5: "van", 6: "truck", 7: "tricycle", 8: "awning-tricycle", 9: "bus", 10: "motor"}


def find_split_dir(root, key):
    """Find a folder whose name contains 'key' ('train'/'val') and that has images/ and annotations/."""
    for dp, dn, _ in os.walk(root):
        if key in os.path.basename(dp).lower() and "images" in dn and "annotations" in dn:
            return dp
    return None


def read_ann(path):
    """Return an (N,8) int array: x,y,w,h,score,cat,trunc,occ."""
    rows = []
    with open(path) as f:
        for line in f:
            p = line.strip().split(",")
            if len(p) >= 8:
                rows.append([int(float(v)) for v in p[:8]])
    return np.array(rows, dtype=np.int64).reshape(-1, 8)


def img_path_of(split_dir, ann_path):
    stem = os.path.splitext(os.path.basename(ann_path))[0]
    return os.path.join(split_dir, "images", stem + ".jpg")


def xywh_to_xyxy(b):
    b = np.asarray(b, dtype=np.float64).reshape(-1, 4)
    return np.stack([b[:, 0], b[:, 1], b[:, 0] + b[:, 2], b[:, 1] + b[:, 3]], 1)


# %% [markdown]
# ## 2. Backbone and dense feature extraction (tiling)
# A large image is split into 518x518 tiles (37x37 patches) with 25% overlap; token grids of overlapping tiles are averaged
# and stitched into one dense, L2-normalised patch-feature map for the whole image.

# %% Backbone
class DinoBackbone:
    """Frozen DINOv2 (HuggingFace). tokens(x) maps (B,3,H,W) in [0,1] -> (B,C,H/14,W/14)."""

    def __init__(self, name="facebook/dinov2-base"):
        from transformers import AutoModel
        self.model = AutoModel.from_pretrained(name).to(DEVICE).eval()
```

```
        if DEVICE == "cuda":
            self.model.half()
        self.dtype = next(self.model.parameters()).dtype
        self.dim = self.model.config.hidden_size
        self.mean = torch.tensor([0.485, 0.456, 0.406], device=DEVICE).view(1, 3, 1, 1)
        self.std = torch.tensor([0.229, 0.224, 0.225], device=DEVICE).view(1, 3, 1, 1)

    @torch.no_grad()
    def tokens(self, x):
        x = ((x - self.mean) / self.std).to(self.dtype)
        out = self.model(pixel_values=x).last_hidden_state[:, 1:]      # drop CLS token
        B, N, C = out.shape
        h, w = x.shape[-2] // PATCH, x.shape[-1] // PATCH
        return out.reshape(B, h, w, C).permute(0, 3, 1, 2).float()


def pil_to_tensor(img):
    arr = np.asarray(img.convert("RGB"))
    return torch.from_numpy(arr).permute(2, 0, 1).float().div(255).to(DEVICE)


def resize_to_patch_multiple(x, scale):
    """Resize (3,H,W) by ~scale so both sides are multiples of 14. Returns (tensor, sx, sy) with sx = newW/W."""
    _, H, W = x.shape
    nh = max(PATCH, int(round(H * scale / PATCH)) * PATCH)
    nw = max(PATCH, int(round(W * scale / PATCH)) * PATCH)
    shrink = (nh < H) or (nw < W)
    y = F.interpolate(x[None], size=(nh, nw), mode="bilinear", align_corners=False, antialias=shrink)[0]
    return y, nw / W, nh / H


def _starts(n, t, st):
    """Tile start indices covering a length-n axis with tiles of size t and stride st."""
    if n <= t:
        return [0], n
    s = list(range(0, n - t + 1, st))
    if s[-1] + t < n:
        s.append(n - t)
    return s, t


@torch.no_grad()
def dense_features(bb, img, tile=518, stride=364, bs=8):
    """Tiled DINOv2 features for a (3,H,W) image (H,W multiples of 14) -> normalised (C,h,w) map."""
    _, H, W = img.shape
    h, w = H // PATCH, W // PATCH
    t, st = tile // PATCH, stride // PATCH
    ys, th = _starts(h, t, st)
    xs, tw = _starts(w, t, st)
    acc = torch.zeros(bb.dim, h, w, device=img.device)
    cnt = torch.zeros(1, h, w, device=img.device)
    coords = [(y, x) for y in ys for x in xs]
    for i in range(0, len(coords), bs):
        chunk = coords[i:i + bs]
        batch = torch.stack([img[:, y * PATCH:(y + th) * PATCH, x * PATCH:(x + tw) * PATCH] for y, x in chunk])
        tok = bb.tokens(batch)
        for (y, x), tk in zip(chunk, tok):
            acc[:, y:y + th, x:x + tw] += tk
            cnt[:, y:y + th, x:x + tw] += 1
    return F.normalize(acc / cnt, dim=0)


class TargetFeatures:
    """Lazily computed, cached feature pyramid of ONE target image.
    Level k has scale 2**(k/2) (half-octave steps). Only requested levels are computed."""

    def __init__(self, bb, pil):
        self.bb = bb
        self.img = pil_to_tensor(pil)
        _, self.H, self.W = self.img.shape
        self.cache, self.cost = {}, {}

    def level(self, k, max_side=4032):
        if k in self.cache:
            return self.cache[k]
        s = 2 ** (k / 2)
        if max(self.H, self.W) * s > max_side or min(self.H, self.W) * s < 3 * PATCH:
            self.cache[k] = None
            return None
        sync(); t0 = time.time()
        x, sx, sy = resize_to_patch_multiple(self.img, s)
        feat = dense_features(self.bb, x)
        sync()
        self.cost[k] = time.time() - t0
        self.cache[k] = dict(feat=feat.to(FEAT_DTYPE), sx=sx, sy=sy)
        return self.cache[k]


# %% [markdown]
# ## 3. Query encoding (orientation-aware)
# The query is a box on a support image. We crop a context window around it, rescale so the object spans about `n_p` patches,
# **rotate** the window by 0/90/180/270 degrees, embed it, and ROI-pool the features inside the box into a small
# `gh x gw` kernel of patch descriptors. Rotating the *image* (not the feature map) is required because ViT features are not rotation-equivariant.

# %% Query encoding
@torch.no_grad()
def extract_query(bb, support, box, rq, rot, min_ctx_px=224, ctx=1.5):
    """support: (3,H,W) tensor; box=(x,y,w,h) in support px; rq: rescale factor; rot: number of 90-degree rotations.
    Returns dict(K=(C,gh,gw) normalised kernel, wh=(w,h) of the box in rescaled+rotated px, rot)."""
    _, H, W = support.shape
    x, y, w, h = [float(v) for v in box]
    cx, cy = x + w / 2, y + h / 2
    half = max(max(w, h) * ctx, (min_ctx_px / 2) / rq)              # context window half-size (original px)
    x0, x1 = int(max(0, math.floor(cx - half))), int(min(W, math.ceil(cx + half)))
    y0, y1 = int(max(0, math.floor(cy - half))), int(min(H, math.ceil(cy + half)))
    crop = support[:, y0:y1, x0:x1]
    mask = torch.zeros(1, y1 - y0, x1 - x0, device=support.device)
    bx0, bx1 = int(max(0, round(x - x0))), int(min(x1 - x0, round(x + w - x0)))
    by0, by1 = int(max(0, round(y - y0))), int(min(y1 - y0, round(y + h - y0)))
    mask[:, by0:max(by1, by0 + 1), bx0:max(bx1, bx0 + 1)] = 1
    crop_s, _, _ = resize_to_patch_multiple(crop, rq)
    mask_s = F.interpolate(mask[None], size=crop_s.shape[-2:], mode="nearest")[0]
    crop_s = torch.rot90(crop_s, rot, dims=(1, 2))
    mask_s = torch.rot90(mask_s, rot, dims=(1, 2))
    tok = bb.tokens(crop_s[None])[0]                                # (C,h,w)
    ys, xs = torch.nonzero(mask_s[0], as_tuple=True)
    mx0, mx1 = xs.min().item(), xs.max().item() + 1
    my0, my1 = ys.min().item(), ys.max().item() + 1
    gh = max(1, int(round((my1 - my0) / PATCH)))
    gw = max(1, int(round((mx1 - mx0) / PATCH)))
    rois = torch.tensor([[0, mx0 / PATCH, my0 / PATCH, mx1 / PATCH, my1 / PATCH]],
                        device=tok.device, dtype=tok.dtype)
    K = roi_align(tok[None], rois, output_size=(gh, gw), spatial_scale=1.0, sampling_ratio=2, aligned=True)[0]
    K = F.normalize(K, dim=0)                                       # per-cell L2 normalisation
    return dict(K=K.to(FEAT_DTYPE), wh=(mx1 - mx0, my1 - my0), rot=rot)


# %% [markdown]
# ## 4. Search: query-scale-adaptive pyramid + rotation-aware template matching + coarse-to-fine refinement

# %% Search
@dataclass
class SearchConfig:
    name: str = "full"
    n_p: int = 3                # object size (in ViT patches) after rescaling the query
    mode: str = "adaptive"      # "single" | "fixed" | "adaptive"  (target-scale policy)
    span: int = 2               # half-octave levels on each side of the centre level
    rotations: tuple = (0, 1, 2, 3)
    refine: bool = True         # coarse-to-fine
    min_score: float = 0.25     # minimum mean cosine similarity for a peak
    topk_per_map: int = 300
    nms_iou: float = 0.5
    max_dets: int = 300
    refine_top: int = 150       # how many candidates get refined
    alpha: float = 0.5          # final = alpha*coarse + (1-alpha)*fine
    max_side: int = 4032        # largest allowed pyramid-level side (px)


def choose_levels(cfg, rq):
    if cfg.mode == "single":
        return [0]                                                  # native resolution only
    if cfg.mode == "fixed":
        return list(range(-cfg.span, cfg.span + 1))                # same levels for every query
    k0 = int(round(2 * math.log2(rq)))                             # level where the query object has n_p patches
    return list(range(k0 - cfg.span, k0 + cfg.span + 1))           # adaptive: centred on the query scale


def match_map(feat, K):
    """Mean cosine similarity of kernel K (C,gh,gw) at every position of feat (C,h,w). Output (h-gh+1, w-gw+1)."""
    gh, gw = K.shape[1:]
    if feat.shape[1] < gh or feat.shape[2] < gw:
        return None
    return F.conv2d(feat[None], (K / (gh * gw))[None])[0, 0].float()


@torch.no_grad()
def coarse_candidates(tf, kernels, levels, cfg):
    """Slide every rotated kernel over every pyramid level; keep local maxima; map to original-image boxes."""
    B, S, KS, RS = [], [], [], []
    for k in levels:
        L = tf.level(k, cfg.max_side)
        if L is None:
            continue
        for q in kernels:
```

```
            sm = match_map(L["feat"], q["K"])
            if sm is None:
                continue
            gh, gw = q["K"].shape[1:]
            mx = F.max_pool2d(sm[None, None], 3, 1, 1)[0, 0]
            ys, xs = torch.nonzero((sm == mx) & (sm > cfg.min_score),
        as_tuple=True)
            if len(ys) == 0:
                continue
            sc = sm[ys, xs]
            if len(sc) > cfg.topk_per_map:
                top = sc.topk(cfg.topk_per_map).indices
                ys, xs, sc = ys[top], xs[top], sc[top]
            cxl = (xs.float() + gw / 2) * PATCH                    # centre in
         level px
            cyl = (ys.float() + gh / 2) * PATCH
            wq, hq = q["wh"]
            B.append(torch.stack([(cxl - wq / 2) / L["sx"], (cyl - hq / 2) /
         L["sy"],
                                  (cxl + wq / 2) / L["sx"], (cyl + hq / 2) /
         L["sy"]], 1))
            S.append(sc)
            KS.append(torch.full_like(sc, k, dtype=torch.long))
            RS.append(torch.full_like(sc, q["rot"], dtype=torch.long))
    if not B:
        return None
    boxes, scores, ks, rots = torch.cat(B), torch.cat(S), torch.cat(KS),
        torch.cat(RS)
    keep = nms(boxes, scores, cfg.nms_iou)[:cfg.max_dets]
    return dict(boxes=boxes[keep], scores=scores[keep], k=ks[keep], rot=rots
        [keep])


@torch.no_grad()
def refine_candidates(bb, tf, support, qbox, rq, dets, cfg):
    """Coarse-to-fine: re-embed a local window around each top candidate at
        2x its level resolution,
    re-match with a 2x-resolution query kernel inside +-2 fine patches of
        the coarse centre, and re-score."""
    fine_q = {r: extract_query(bb, support, qbox, 2 * rq, r) for r in set(
        dets["rot"].tolist())}
    boxes, scores = dets["boxes"].clone(), dets["scores"].clone()
    new_scores = scores * cfg.alpha                                #
        candidates that are not refined are demoted
    m = min(len(scores), cfg.refine_top)
    for i in range(m):
        k, r = int(dets["k"][i]), int(dets["rot"][i])
        s_f = 2 * (2 ** (k / 2))                                   # fine
        scale = 2x the coarse level scale
        x0, y0, x1, y1 = boxes[i].tolist()
        cx, cy = (x0 + x1) / 2, (y0 + y1) / 2
        half = max(max(x1 - x0, y1 - y0) * 1.5, 64 / s_f)          # window ~3
        x the object (at least ~128 px at fine scale)
        wx0, wx1 = int(max(0, math.floor(cx - half))), int(min(tf.W, math.
        ceil(cx + half)))
        wy0, wy1 = int(max(0, math.floor(cy - half))), int(min(tf.H, math.
        ceil(cy + half)))
        if wx1 - wx0 < 4 or wy1 - wy0 < 4:
            continue
        win_s, sx, sy = resize_to_patch_multiple(tf.img[:, wy0:wy1, wx0:wx1
        ], s_f)
        feat = F.normalize(bb.tokens(win_s[None])[0], dim=0).to(FEAT_DTYPE)
        K = fine_q[r]["K"]
        sm = match_map(feat, K)
        if sm is None:
            continue
        gh, gw = K.shape[1:]
        # restrict to positions near the coarse centre (avoids jumping to a
        neighbouring instance)
        ecx, ecy = (cx - wx0) * sx, (cy - wy0) * sy
        px = (torch.arange(sm.shape[1], device=sm.device).float() + gw / 2)
        * PATCH
        py = (torch.arange(sm.shape[0], device=sm.device).float() + gh / 2)
        * PATCH
        near = ((py[:, None] - ecy).abs() <= 2 * PATCH) & ((px[None, :] -
        ecx).abs() <= 2 * PATCH)
        if not near.any():
            continue
        sm = sm.masked_fill(~near, -1.0)
        idx = int(sm.argmax())
        yy, xx = divmod(idx, sm.shape[1])
        fs = float(sm[yy, xx])
        cxl, cyl = (xx + gw / 2) * PATCH, (yy + gh / 2) * PATCH
        wq, hq = fine_q[r]["wh"]
        boxes[i] = torch.tensor([wx0 + (cxl - wq / 2) / sx, wy0 + (cyl - hq
        / 2) / sy,
                                 wx0 + (cxl + wq / 2) / sx, wy0 + (cyl + hq
        / 2) / sy], device=boxes.device)
        new_scores[i] = cfg.alpha * scores[i] + (1 - cfg.alpha) * fs
    keep = nms(boxes, new_scores, cfg.nms_iou)[:cfg.max_dets]
    return dict(boxes=boxes[keep], scores=new_scores[keep], k=dets["k"][keep
        ], rot=dets["rot"][keep])


@torch.no_grad()
def run_search(bb, tf, support, qbox, cfg):
    """Full pipeline for one query on one target image. Returns dict(boxes,
        scores, k, rot, time)."""
    before = set(tf.cost.keys())
    sync(); t0 = time.time()
    w, h = qbox[2], qbox[3]
    rq = cfg.n_p * PATCH / max(w, h)                               # rescale
        so the query object spans ~n_p patches
    kernels = [extract_query(bb, support, qbox, rq, r) for r in cfg.
        rotations]
    levels = choose_levels(cfg, rq)
    dets = coarse_candidates(tf, kernels, levels, cfg)
    if dets is not None and cfg.refine:
        dets = refine_candidates(bb, tf, support, qbox, rq, dets, cfg)
    sync()
    wall = time.time() - t0
    # fair timing: add the cost of levels that were already cached by an
        earlier configuration
    reused = sum(tf.cost[k] for k in levels if k in tf.cost and k in before)
    if dets is None:
        return dict(boxes=np.zeros((0, 4)), scores=np.zeros(0), k=np.zeros
        (0), rot=np.zeros(0), time=wall + reused)
    return dict(boxes=dets["boxes"].cpu().numpy(), scores=dets["scores"].cpu
        ().numpy(),
                k=dets["k"].cpu().numpy(), rot=dets["rot"].cpu().numpy(),
        time=wall + reused)


# %% [markdown]
# ## 5. Evaluation (episodic one-shot protocol)
# * **Episode** = (target image from VisDrone *val*, category c, query = one
        clean instance of c from a *different* VisDrone *train* image).
# * Task: find all instances of c in the target image. Ground truth =
        category-c boxes with score=1; detections inside "ignored regions"
        are dropped.
# * Metrics: **AP@0.5** per episode (all-point interpolation), **Recall@100
        **, and **runtime** per episode. We report the mean over episodes,
        per config, per class, and per object-size bucket.
# * All configurations see exactly the same episodes (fixed seed).

# %% Metrics (pure numpy)
def iou_matrix(a, b):
    a, b = np.asarray(a, float).reshape(-1, 4), np.asarray(b, float).reshape
        (-1, 4)
    ix0 = np.maximum(a[:, None, 0], b[None, :, 0]); iy0 = np.maximum(a[:,
        None, 1], b[None, :, 1])
    ix1 = np.minimum(a[:, None, 2], b[None, :, 2]); iy1 = np.minimum(a[:,
        None, 3], b[None, :, 3])
    inter = np.clip(ix1 - ix0, 0, None) * np.clip(iy1 - iy0, 0, None)
    aa = (a[:, 2] - a[:, 0]) * (a[:, 3] - a[:, 1]); ab = (b[:, 2] - b[:, 0])
        * (b[:, 3] - b[:, 1])
    return inter / (aa[:, None] + ab[None, :] - inter + 1e-9)


def eval_episode(det_boxes, det_scores, gt_xyxy, ignore_xyxy, thr=0.5, K
        =100):
    """Return (AP, Recall@K). det_boxes xyxy; gt/ignore xyxy."""
    n_gt = len(gt_xyxy)
    if n_gt == 0:
        return None
    det_boxes = np.asarray(det_boxes, float).reshape(-1, 4)
    det_scores = np.asarray(det_scores, float).reshape(-1)
    if len(ignore_xyxy) and len(det_boxes):                        # drop
        detections centred in ignored regions
        c = np.stack([(det_boxes[:, 0] + det_boxes[:, 2]) / 2, (det_boxes[:,
        1] + det_boxes[:, 3]) / 2], 1)
        inside = np.zeros(len(det_boxes), bool)
        for g in ignore_xyxy:
            inside |= (c[:, 0] >= g[0]) & (c[:, 0] <= g[2]) & (c[:, 1] >= g
        [1]) & (c[:, 1] <= g[3])
        det_boxes, det_scores = det_boxes[~inside], det_scores[~inside]
    if len(det_boxes) == 0:
        return 0.0, 0.0
    order = np.argsort(-det_scores)
    det_boxes = det_boxes[order]
    iou = iou_matrix(det_boxes, gt_xyxy)
    matched = np.zeros(n_gt, bool)
    tp = np.zeros(len(det_boxes))
    for i in range(len(det_boxes)):
        j = int(np.argmax(iou[i]))
        if iou[i, j] >= thr and not matched[j]:
            tp[i] = 1; matched[j] = True
    ctp, cfp = np.cumsum(tp), np.cumsum(1 - tp)
    rec, prec = ctp / n_gt, ctp / np.maximum(ctp + cfp, 1e-9)
    mrec, mpre = np.concatenate([[0], rec, [1]]), np.concatenate([[0], prec,
        [0]])
    for i in range(len(mpre) - 2, -1, -1):                         #
        precision envelope
        mpre[i] = max(mpre[i], mpre[i + 1])
    idx = np.where(mrec[1:] != mrec[:-1])[0]
    ap = float(np.sum((mrec[idx + 1] - mrec[idx]) * mpre[idx + 1]))
    return ap, float(tp[:K].sum() / n_gt)


# %% Episode construction
def build_episodes(train_dir, val_dir, n_images=60, cats_per_image=2, min_gt
        =3, seed=0):
    rng = random.Random(seed)
    pool = defaultdict(list)                                       # category
        -> [(support_img, box_xywh)]
    train_anns = sorted(glob.glob(os.path.join(train_dir, "annotations", "*.
        txt")))[::5]
    for ap_ in train_anns:
        a = read_ann(ap_)
        for x, y, w, h, sc, c, tr, oc in a:
            if c in CLASSES and sc == 1 and tr == 0 and oc == 0 and min(w, h
        ) >= 12 and max(w, h) <= 300:
                pool[int(c)].append((img_path_of(train_dir, ap_), (int(x),
        int(y), int(w), int(h))))
    val_anns = sorted(glob.glob(os.path.join(val_dir, "annotations", "*.txt"
        )))
    rng.shuffle(val_anns)
    episodes, n_img = [], 0
    for ap_ in val_anns:
        a = read_ann(ap_)
        cats = [c for c in CLASSES if ((a[:, 5] == c) & (a[:, 4] == 1)).sum
        () >= min_gt and len(pool[c]) > 0]
```

```
        if not cats:
            continue
        rng.shuffle(cats)
        for c in cats[:cats_per_image]:
            sup, qbox = rng.choice(pool[c])
            gt = a[(a[:, 5] == c) & (a[:, 4] == 1)]
            episodes.append(dict(ann=ap_, img=img_path_of(val_dir, ap_), cat=c, support=sup, qbox=qbox,
                                 gt_med=float(np.median(np.maximum(gt[:, 2], gt[:, 3]))), n_gt=len(gt)))
        n_img += 1
        if n_img >= n_images:
            break
    print(f"{len(episodes)} episodes on {n_img} images")
    return episodes


# $$ [markdown]
# ## 6. OWL-ViT image-guided baseline (tiled)
# Baseline from Minderer et al. (ECCV 2022): image-conditioned detection. Run on 768x768 tiles because the model resizes its input to 768x768.

# $$ OWL-ViT baseline
class OwlBaseline:
    def __init__(self, name="google/owlvit-base-patch32"):
        from transformers import OwlViTProcessor, OwlViTForObjectDetection
        self.proc = OwlViTProcessor.from_pretrained(name)
        self.model = OwlViTForObjectDetection.from_pretrained(name).to(DEVICE).eval()

    @torch.no_grad()
    def search(self, target_pil, support_pil, qbox, scale=1.0, tile=768, stride=576, thr=0.05):
        x, y, w, h = qbox
        m = 0.15 * max(w, h)                                   # small context margin around the query crop
        query = support_pil.crop((max(0, x - m), max(0, y - m), x + w + m, y + h + m))
        img = target_pil if scale == 1.0 else target_pil.resize(
            (int(target_pil.width * scale), int(target_pil.height * scale)), Image.BILINEAR)
        W, H = img.size
        xs, tw = _starts(W, tile, stride)
        ys, th = _starts(H, tile, stride)
        sync(); t0 = time.time()
        B, S = [], []
        for ty in ys:
            for tx in xs:
                t_img = img.crop((tx, ty, tx + tw, ty + th))
                inp = self.proc(images=t_img, query_images=query, return_tensors="pt")
                inp = {k: v.to(DEVICE) for k, v in inp.items()}
                out = self.model.image_guided_detection(**inp)
                res = self.proc.post_process_image_guided_detection(
                    outputs=out, threshold=thr, nms_threshold=0.3,
                    target_sizes=torch.tensor([[th, tw]], device=out.logits.device))[0]
                b = res["boxes"].cpu().numpy() + np.array([tx, ty, tx, ty])
                B.append(b / scale); S.append(res["scores"].cpu().numpy())
        sync()
        if not B or sum(len(s) for s in S) == 0:
            return dict(boxes=np.zeros((0, 4)), scores=np.zeros(0), time=time.time() - t0)
        boxes, scores = np.concatenate(B), np.concatenate(S)
        keep = nms(torch.tensor(boxes, dtype=torch.float32), torch.tensor(scores, dtype=torch.float32), 0.5)[:300].numpy()
        return dict(boxes=boxes[keep], scores=scores[keep], time=time.time() - t0)


# $$ [markdown]
# ## 7. Run the benchmark

# $$ Benchmark runner
def run_benchmark(bb, episodes, configs, owl=None, owl_scales=(1.0, 2.0)):
    records = []
    by_img = defaultdict(list)
    for ep in episodes:
        by_img[ep["img"]].append(ep)
    owl_ok = owl is not None
    for n, (img_path, eps) in enumerate(by_img.items()):
        pil = Image.open(img_path).convert("RGB")
        tf = TargetFeatures(bb, pil)                           # levels are shared (cached) across configs
        for ep in eps:
            a = read_ann(ep["ann"])
            gt = xywh_to_xyxy(a[(a[:, 5] == ep["cat"]) & (a[:, 4] == 1), :4])
            ign = xywh_to_xyxy(a[a[:, 5] == 0, :4])
            sup_pil = Image.open(ep["support"]).convert("RGB")
            sup_t = pil_to_tensor(sup_pil)
            base = dict(img=os.path.basename(img_path), cat=CLASSES[ep["cat"]], n_gt=ep["n_gt"], gt_med=ep["gt_med"])
            for cfg in configs:
                r = run_search(bb, tf, sup_t, ep["qbox"], cfg)
                m = eval_episode(r["boxes"], r["scores"], gt, ign)
                records.append(dict(base, method=cfg.name, ap=m[0], r100=m[1], time=r["time"]))
            if owl_ok:
                try:
                    for s in owl_scales:
                        r = owl.search(pil, sup_pil, ep["qbox"], scale=s)
                        m = eval_episode(r["boxes"], r["scores"], gt, ign)
                        records.append(dict(base, method=f"OWL-ViT tiled ({s:g}x)", ap=m[0], r100=m[1], time=r["time"]))
                except Exception as e:                         # API differences should not kill the whole run
                    print("OWL-ViT baseline disabled:", repr(e)); owl_ok = False
        del tf
        if DEVICE == "cuda":
            torch.cuda.empty_cache()
        if (n + 1) % 5 == 0:
            print(f"{n + 1}/{len(by_img)} images done")
    return pd.DataFrame(records)


def summarize(df):
    """Main table, per-size and per-class tables (values in %, runtime in s)."""
    order = list(dict.fromkeys(df["method"]))
    main = df.groupby("method").agg(mAP50=("ap", "mean"), R100=("r100", "mean"), time_s=("time", "mean")).reindex(order)
    main[["mAP50", "R100"]] *= 100
    df = df.copy()
    df["size"] = pd.cut(df["gt_med"], [0, 32, 64, 1e9], labels=["small (<32 px)", "medium (32-64px)", "large (>64px)"])
    by_size = (df.pivot_table(index="method", columns="size", values="ap", aggfunc="mean", observed=False) * 100).reindex(order)
    by_cls = (df.pivot_table(index="method", columns="cat", values="ap", aggfunc="mean") * 100).reindex(order)
    return main.round(2), by_size.round(2), by_cls.round(2)


# $$ Smoke test (SMALL: 6 images) -- run this first, then increase N_IMAGES
ROOT = "/kaggle/input" if os.path.exists("/kaggle/input") else "."
TRAIN_DIR, VAL_DIR = find_split_dir(ROOT, "train"), find_split_dir(ROOT, "val")
print("train:", TRAIN_DIR, "\nval:", VAL_DIR)
assert TRAIN_DIR and VAL_DIR, "Could not find VisDrone train/val folders (each needs images/ and annotations/). Check the Add Input dataset."

bb = DinoBackbone("facebook/dinov2-base")

CONFIGS = [
    SearchConfig(name="A: single scale",           mode="single",   rotations=(0,),          refine=False),
    SearchConfig(name="B: fixed pyramid",          mode="fixed",    rotations=(0,),          refine=False),
    SearchConfig(name="C: + adaptive scales",      mode="adaptive", rotations=(0,),          refine=False),
    SearchConfig(name="D: + rotation",             mode="adaptive", rotations=(0, 1, 2, 3),  refine=False),
    SearchConfig(name="E: + coarse-to-fine (full)", mode="adaptive", rotations=(0, 1, 2, 3), refine=True),
]
episodes = build_episodes(TRAIN_DIR, VAL_DIR, n_images=6)
df_smoke = run_benchmark(bb, episodes, CONFIGS, owl=None)
print(summarize(df_smoke)[0])

# $$ Full benchmark (increase N_IMAGES as far as your Kaggle time allows; 60 images ~ 100 episodes)
N_IMAGES = 60
episodes = build_episodes(TRAIN_DIR, VAL_DIR, n_images=N_IMAGES)
owl = None
try:
    owl = OwlBaseline()
except Exception as e:
    print("OWL-ViT could not be loaded:", repr(e))
df = run_benchmark(bb, episodes, CONFIGS, owl=owl)
df.to_csv(f"{OUT_DIR}/all_episodes.csv", index=False)
main, by_size, by_cls = summarize(df)
print(main, "\n", by_size, "\n", by_cls)
for name, t in [("main", main), ("by_size", by_size), ("by_class", by_cls)]:
    t.to_csv(f"{OUT_DIR}/{name}.csv")
    open(f"{OUT_DIR}/{name}.tex", "w").write(t.to_latex(float_format="%.2f"))   # paste into the CVPR template

# $$ Sensitivity to the query-object size n_p (hyper-parameter study for the report)
rows = []
for n_p in (2, 3, 4):
    cfg = SearchConfig(name=f"full, n_p={n_p}", n_p=n_p)
    d = run_benchmark(bb, episodes[:40], [cfg], owl=None)
    rows.append(summarize(d)[0])
sens = pd.concat(rows); print(sens); sens.to_csv(f"{OUT_DIR}/sensitivity_np.csv")

# $$ [markdown]
# ## 8. Qualitative results (figures for the report)

# $$ Visualisation
def draw(ax, pil, boxes, scores, gt=None, topn=40, title=""):
    ax.imshow(pil); ax.set_title(title, fontsize=9); ax.axis("off")
    if gt is not None:
        for b in gt:
            ax.add_patch(mpatches.Rectangle((b[0], b[1]), b[2] - b[0], b[3] - b[1], fill=False, ec="lime", lw=1))
    for b, s in list(zip(boxes, scores))[:topn]:
        ax.add_patch(mpatches.Rectangle((b[0], b[1]), b[2] - b[0], b[3] - b[1], fill=False, ec="red", lw=1.2))


def qualitative(bb, episodes, cfg, n=4, topn=40, save=True):
    fig, axes = plt.subplots(n, 2, figsize=(14, 5 * n))
    for row, ep in zip(axes, episodes[:n]):
        pil = Image.open(ep["img"]).convert("RGB")
        tf = TargetFeatures(bb, pil)
        sup_pil = Image.open(ep["support"]).convert("RGB")
        a = read_ann(ep["ann"])
        gt = xywh_to_xyxy(a[(a[:, 5] == ep["cat"]) & (a[:, 4] == 1), :4])
```

```
        r = run_search(bb, tf, pil_to_tensor(sup_pil), ep["qbox"], cfg)
        x, y, w, h = ep["qbox"]
        row[0].imshow(sup_pil.crop((x, y, x + w, y + h))); row[0].axis("off")
        row[0].set_title(f"query ({CLASSES[ep['cat']]})")
        draw(row[1], pil, r["boxes"], r["scores"], gt, topn, f"red=detections (top {topn}), green=GT")
    plt.tight_layout()
    if save:
        plt.savefig(f"{OUT_DIR}/qualitative.png", dpi=130, bbox_inches="tight")
    plt.show()


qualitative(bb, episodes, CONFIGS[-1], n=4)


# $$ [markdown]
# ## 9. Held-out tuning, final ablation and statistics (cells added after the first full run)

# $$ Tune n_p on held-out episodes (images disjoint from the evaluation episodes)
test_imgs = {e["img"] for e in episodes}
tune_eps = [e for e in build_episodes(TRAIN_DIR, VAL_DIR, n_images=40, seed=1) if e["img"] not in test_imgs]
rows = []
for n_p in (3, 4, 5, 6):
    cfgs = [SearchConfig(name=f"fixed, n_p={n_p}", n_p=n_p, mode="fixed", rotations=(0,), refine=False),
            SearchConfig(name=f"adaptive, n_p={n_p}", n_p=n_p, mode="adaptive", rotations=(0,), refine=False),
            SearchConfig(name=f"full, n_p={n_p}", n_p=n_p)]
    rows.append(summarize(run_benchmark(bb, tune_eps, cfgs, owl=None))[0])
tune = pd.concat(rows); print(tune); tune.to_csv(f"{OUT_DIR}/tune_np.csv")

# $$ Final ablation with the chosen n_p (A-E), OWL-ViT rows are re-used from the first full run
from dataclasses import replace
BEST_NP = 4   # chosen after looking at the tuning table
CONFIGS_FINAL = [replace(c, n_p=BEST_NP) for c in CONFIGS]
df_final = run_benchmark(bb, episodes, CONFIGS_FINAL, owl=None)
df_all = pd.concat([df_final, df[df.method.str.startswith("OWL")]])
df_all.to_csv(f"{OUT_DIR}/all_episodes_final.csv", index=False)
main, by_size, by_cls = summarize(df_all); print(main, by_size)

# $$ Paired bootstrap confidence intervals (episode level) for consecutive ablation steps
df = pd.read_csv(f"{OUT_DIR}/all_episodes_final.csv")
print(len(df), df.method.unique())
df["ep"] = df["img"] + "|" + df["cat"]
P = df.pivot(index="ep", columns="method", values="ap") * 100
rng = np.random.default_rng(0)
def paired(a, b, n=10000):
    d = (P[a] - P[b]).dropna().values
    boots = rng.choice(d, (n, len(d))).mean(1)
    return d.mean(), *np.percentile(boots, [2.5, 97.5])
for a, b in [("B: fixed pyramid", "A: single scale"), ("C: + adaptive scales", "B: fixed pyramid"),
             ("D: + rotation", "C: + adaptive scales"), ("E: + coarse-to-fine (full)", "D: + rotation"),
             ("E: + coarse-to-fine (full)", "B: fixed pyramid")]:
    m, lo, hi = paired(a, b); print(f"{a} - {b}: {m:+.2f}  95% CI [{lo:+.2f}, {hi:+.2f}]")

# $$ Extra configurations F, G (added AFTER seeing the A-E results: exploratory) + paired CIs for AP and Recall@100
CONFIGS_EXTRA = [
    replace(SearchConfig(name="F: fixed + coarse-to-fine", mode="fixed", rotations=(0,), refine=True), n_p=BEST_NP),
    replace(SearchConfig(name="G: fixed + rotation + coarse-to-fine", mode="fixed", rotations=(0, 1, 2, 3), refine=True), n_p=BEST_NP),
]
df_extra = run_benchmark(bb, episodes, CONFIGS_EXTRA, owl=None)
df_all2 = pd.concat([df_all, df_extra])
df_all2.to_csv(f"{OUT_DIR}/all_episodes_final2.csv", index=False)
main, by_size, by_cls = summarize(df_all2); print(main); print(by_size)

d = df_all2.copy(); d["ep"] = d["img"] + "|" + d["cat"]
rng = np.random.default_rng(0)
def paired(metric, a, b, n=10000):
    P = d.pivot(index="ep", columns="method", values=metric) * 100
    x = (P[a] - P[b]).dropna().values
    boots = rng.choice(x, (n, len(x))).mean(1)
    return x.mean(), *np.percentile(boots, [2.5, 97.5])
E, B = "E: + coarse-to-fine (full)", "B: fixed pyramid"
F_, G_ = "F: fixed + coarse-to-fine", "G: fixed + rotation + coarse-to-fine"
for metric in ("ap", "r100"):
    print("---", metric)
    for a, b in [(F_, B), (F_, E), (G_, F_), (E, B)]:
        m, lo, hi = paired(metric, a, b); print(f"{a} - {b}: {m:+.2f}  95% CI [{lo:+.2f}, {hi:+.2f}]")

# $$ Qualitative figures for the final method G (best / worst episodes by AP)
FINAL = CONFIGS_EXTRA[1]
dG = df_all2[df_all2.method == FINAL.name].sort_values("ap")
def find(r):
    return next(e for e in episodes if os.path.basename(e["img"]) == r.img and CLASSES[e["cat"]] == r.cat)
best  = [find(r) for r in dG.tail(3).itertuples()]
worst = [find(r) for r in dG.head(3).itertuples()]
qualitative(bb, best, FINAL, n=3)             # saves qualitative.png
os.replace(f"{OUT_DIR}/qualitative.png", f"{OUT_DIR}/qual_success.png")
qualitative(bb, worst, FINAL, n=3)
os.replace(f"{OUT_DIR}/qualitative.png", f"{OUT_DIR}/qual_failure.png")

# $$ AP at looser IoU thresholds for the final method G (localisation diagnostic)
rows = []
by_img = defaultdict(list)
for ep in episodes: by_img[ep["img"]].append(ep)
for img_path, eps in by_img.items():
    tf = TargetFeatures(bb, Image.open(img_path).convert("RGB"))
    for ep in eps:
        a = read_ann(ep["ann"])
        gt = xywh_to_xyxy(a[(a[:, 5] == ep["cat"]) & (a[:, 4] == 1), :4])
        ign = xywh_to_xyxy(a[a[:, 5] == 0, :4])
        sup = pil_to_tensor(Image.open(ep["support"]).convert("RGB"))
        r = run_search(bb, tf, sup, ep["qbox"], FINAL)
        rows.append({f"AP@{t}": eval_episode(r["boxes"], r["scores"], gt, ign, thr=t)[0] for t in (0.1, 0.3, 0.5)})
    del tf; torch.cuda.empty_cache()
print((pd.DataFrame(rows).mean() * 100).round(2))
```

C.2. Interactive demo (Gradio cell)

```
# $$ Gradio demo (paste as ONE new cell at the bottom of the notebook)
# Needs: the cells above that define DinoBackbone, TargetFeatures, SearchConfig, run_search (sections 1-4).
# Run first in a separate cell:  !pip -q install gradio
# How to use: upload a drone image, click TWO opposite corners of the object, press "Search".

import hashlib
from dataclasses import replace

import numpy as np
from PIL import Image, ImageDraw

import types
vc = types.SimpleNamespace(DinoBackbone=DinoBackbone, TargetFeatures=TargetFeatures, SearchConfig=SearchConfig,
                           run_search=run_search, pil_to_tensor=pil_to_tensor)
if "bb" not in globals():                       # reuse the backbone already loaded in the notebook
    bb = DinoBackbone("facebook/dinov2-base")

N_P = 4                                         # query object size in patches (value used in the report)
ROT_LABEL = {0: "0", 1: "90", 2: "180", 3: "270"}
POLICIES = {"Fixed pyramid (evaluated, config G)": "fixed",
            "Adaptive to query size (not evaluated for same-image queries)": "adaptive"}

_state = {"bb": bb, "key": None, "tf": None}


# ---------------------------------------------------------------------------- helpers (no GUI)
def get_backbone():
    """Load DINOv2 once (first call takes a while)."""
    if _state["bb"] is None:
        _state["bb"] = vc.DinoBackbone("facebook/dinov2-base")
    return _state["bb"]


def get_target(img):
    """Feature pyramid of the target image, cached so repeated queries on the same image are fast."""
    key = hashlib.md5(img.tobytes()).hexdigest() + str(img.size)
    if _state["key"] != key:
        _state["tf"] = vc.TargetFeatures(get_backbone(), img)
        _state["key"] = key
    return _state["tf"]


def click_logic(corner, x, y):
    """Two-click box selection. Returns (new_corner, box_or_None); box = (x, y, w, h)."""
    if corner is None:
        return (int(x), int(y)), None
    x0, y0 = min(corner[0], int(x)), min(corner[1], int(y))
    x1, y1 = max(corner[0], int(x)), max(corner[1], int(y))
    return None, (x0, y0, x1 - x0, y1 - y0)


def support_from_crop(crop):
    """Turn an uploaded query crop into a (support image, box) pair by padding it with its mean colour.
    Less reliable than marking the object on a real image, because the crop has no real context."""
    c = np.asarray(crop.convert("RGB"))
    h, w = c.shape[:2]
    pad = max(h, w)
    canvas = np.empty((h + 2 * pad, w + 2 * pad, 3), np.uint8)
    canvas[:] = c.reshape(-1, 3).mean(0).astype(np.uint8)
    canvas[pad:pad + h, pad:pad + w] = c
    return Image.fromarray(canvas), (pad, pad, w, h)


def merge_overlaps(boxes, scores, iou_thr=0.3, contain_thr=0.6):
    """DISPLAY-ONLY clean-up (not part of the evaluated pipeline): boxes must be sorted by score, best first.
    A box is dropped if it overlaps a kept box by IoU > iou_thr, or if it is mostly contained in / contains a kept box."""
    keep = []
```

```
    for i in range(len(boxes)):
        a, ok = boxes[i], True
        for j in keep:
            b = boxes[j]
            iw = min(a[2], b[2]) - max(a[0], b[0]); ih = min(a[3], b[3]) - max(a[1], b[1])
            inter = max(iw, 0) * max(ih, 0)
            aa, ab = (a[2] - a[0]) * (a[3] - a[1]), (b[2] - b[0]) * (b[3] - b[1])
            if inter / (aa + ab - inter + 1e-9) > iou_thr or inter / (min(aa, ab) + 1e-9) > contain_thr:
                ok = False
                break
        if ok:
            keep.append(i)
    return keep


def draw_boxes(img, boxes, scores, rots, query_box=None):
    """Return a copy of img with detections (red) and the query box (blue)."""
    out = img.convert("RGB").copy()
    d = ImageDraw.Draw(out)
    lw = max(2, max(out.size) // 700)
    for (x0, y0, x1, y1), s, r in zip(boxes, scores, rots):
        d.rectangle([x0, y0, x1, y1], outline=(255, 40, 40), width=lw)
        d.text((x0 + 2, max(0, y0 - 11)), f"{s:.2f}/{ROT_LABEL[int(r)]}", fill=(255, 255, 0))
    if query_box is not None:
        x, y, w, h = query_box
        d.rectangle([x, y, x + w, y + h], outline=(40, 120, 255), width=lw + 1)
    return out


def preview_query(img, corner=None, box=None):
    """Show the current corner / box on a copy of the image so the user sees what was selected."""
    out = img.convert("RGB").copy()
    d = ImageDraw.Draw(out)
    lw = max(2, max(out.size) // 700)
    if box is not None and box[2] > 0 and box[3] > 0:
        x, y, w, h = box
        d.rectangle([x, y, x + w, y + h], outline=(40, 120, 255), width=lw + 1)
    if corner is not None:
        r = 5 * lw
        d.ellipse([corner[0] - r, corner[1] - r, corner[0] + r, corner[1] + r], outline=(40, 120, 255), width=lw)
    return out


def do_search(img, mode, bx, by, bw, bh, crop, policy, use_rot, use_refine, min_score, max_results, clean=True):
    """Run the search. Returns (annotated image or None, table rows, status text)."""
    if img is None:
        return None, [], "Please upload a drone image first."
    img = img.convert("RGB")
    if mode == "Upload a query crop":
        if crop is None:
            return None, [], "Upload a query crop, or switch to 'Mark box on the drone image'."
        support, qbox = support_from_crop(crop)
        query_box_for_drawing = None
    else:
        qbox = (int(bx), int(by), int(bw), int(bh))
        if qbox[2] < 4 or qbox[3] < 4:
            return None, [], "The query box is too small. Click two opposite corners of the object (or set w, h >= 4)."
        W, H = img.size
        if qbox[0] < 0 or qbox[1] < 0 or qbox[0] + qbox[2] > W or qbox[1] + qbox[3] > H:
            return None, [], "The query box is outside the image."
        support, query_box_for_drawing = img, qbox

    cfg = replace(vc.SearchConfig(name="demo", n_p=N_P, mode=POLICIES[policy]),
                  rotations=(0, 1, 2, 3) if use_rot else (0,), refine=bool(use_refine))
    tf = get_target(img)
    res = vc.run_search(get_backbone(), tf, vc.pil_to_tensor(support), qbox, cfg)

    boxes, scores, rots = np.asarray(res["boxes"]), np.asarray(res["scores"]), np.asarray(res["rot"]).astype(int)
    W, H = img.size
    if len(boxes):
        boxes = np.stack([np.clip(boxes[:, 0], 0, W), np.clip(boxes[:, 1], 0, H),
                          np.clip(boxes[:, 2], 0, W), np.clip(boxes[:, 3], 0, H)], 1)
    order = np.argsort(-scores)
    order = [i for i in order if scores[i] >= min_score]
    if clean and len(order):
        order = [order[k] for k in merge_overlaps(boxes[order], scores[order])]
    order = order[:int(max_results)]
    boxes, scores, rots = boxes[order], scores[order], rots[order]

    annotated = draw_boxes(img, boxes, scores, rots, query_box_for_drawing)
    rows = [[i + 1, int(b[0]), int(b[1]), int(b[2] - b[0]), int(b[3] - b[1]), round(float(s), 3), ROT_LABEL[int(r)]]
            for i, (b, s, r) in enumerate(zip(boxes, scores, rots))]
    status = (f"Showing **{len(rows)}** matches (score >= {min_score:.2f}{', overlaps merged' if clean else ''}) in {res['time']:.1f} s. "
              "In 'Mark box' mode the query object itself is usually the top match.")
    return annotated, rows, status


# ---------------------------------------------------------------------------
# GUI
def build_ui():
    import gradio as gr

    def on_click(img, corner, evt: gr.SelectData):
        if img is None:
            return corner, gr.update(), gr.update(), gr.update(), gr.update(), None
        x, y = evt.index[0], evt.index[1]
        corner, box = click_logic(corner, x, y)
        if box is None:   # first corner
            return corner, gr.update(), gr.update(), gr.update(), gr.update(), preview_query(img, corner=corner)
        return corner, box[0], box[1], box[2], box[3], preview_query(img, box=box)

    def on_reset(img):
        return None, 0, 0, 0, 0, (img if img is None else img.convert("RGB"))

    def on_numbers(img, bx, by, bw, bh):
        return None if img is None else preview_query(img, box=(bx, by, bw, bh))

    with gr.Blocks(title="Visual Object Search in Drone Images") as demo:
        gr.Markdown(
            "# Visual Object Search in Drone Images\n"
            "Find every instance of an object you point at, with **no training** (frozen DINOv2 + tiled feature "
            "pyramid + rotated queries + coarse-to-fine refinement).  \n"
            "**1.** Upload a drone image. **2.** Click two opposite corners of the object. **3.** Press *Search*.  \n"
            "*The first search on a new image is slower (it builds the feature pyramid). Output box size is taken "
            "from your query box, so tight query boxes work best.*")
        corner = gr.State(None)
        with gr.Row():
            with gr.Column():
                img = gr.Image(type="pil", label="1) Drone image  (click two corners of the object)", height=420)
                qprev = gr.Image(label="Selected query box", interactive=False, height=300)
                with gr.Row():
                    bx = gr.Number(label="x", value=0, precision=0)
                    by = gr.Number(label="y", value=0, precision=0)
                    bw = gr.Number(label="w", value=0, precision=0)
                    bh = gr.Number(label="h", value=0, precision=0)
                reset = gr.Button("Reset box")
                mode = gr.Radio(["Mark box on the drone image", "Upload a query crop"],
                                value="Mark box on the drone image", label="Query source")
                crop = gr.Image(type="pil", label="Query crop (only used in 'Upload a query crop' mode)", height=160)
            with gr.Column():
                with gr.Accordion("Settings", open=False):
                    policy = gr.Radio(list(POLICIES), value=list(POLICIES)[0], label="Scale policy")
                    use_rot = gr.Checkbox(value=True, label="Rotation-aware query (0/90/180/270 deg)")
                    use_refine = gr.Checkbox(value=True, label="Coarse-to-fine refinement")
                    clean = gr.Checkbox(value=True, label="Merge overlapping boxes (display only; not part of the evaluated method)")
                    min_score = gr.Slider(0.2, 0.9, value=0.45, step=0.01, label="Minimum score to display")
                    max_results = gr.Slider(1, 200, value=40, step=1, label="Max matches to display")
                run = gr.Button("Search", variant="primary")
                status = gr.Markdown()
                out_img = gr.Image(label="Result (red = matches with score/rotation, blue = query)", height=520)
                table = gr.Dataframe(headers=["#", "x", "y", "w", "h", "score", "rot (deg)"], interactive=False)

        img.select(on_click, [img, corner], [corner, bx, by, bw, bh, qprev])
        img.upload(on_reset, [img], [corner, bx, by, bw, bh, qprev])
        reset.click(on_reset, [img], [corner, bx, by, bw, bh, qprev])
        for n in (bx, by, bw, bh):
            n.change(on_numbers, [img, bx, by, bw, bh], [qprev])
        run.click(do_search, [img, mode, bx, by, bw, bh, crop, policy, use_rot, use_refine, min_score, max_results, clean],
                  [out_img, table, status])
    return demo


demo = build_ui()
demo.queue().launch(share=True)
```